

Celebal Summer Internship 2026

Week 2 Assignment

SQL-Based Data Analysis using MySQL

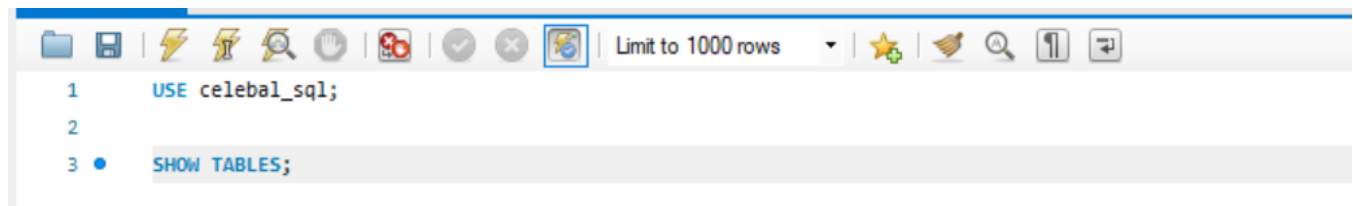
Objective

Perform SQL-based data analysis using filtering, aggregation, joins, sorting, grouping, and business queries on the given e-commerce database.

Database Verification

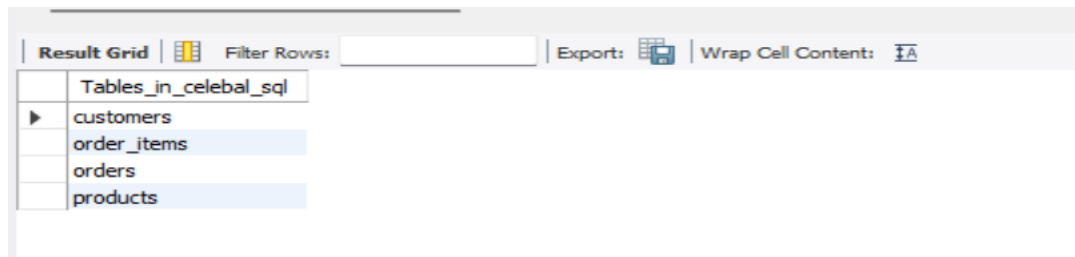
Show Tables

Query Screenshot



```
1 USE celebal_sql;
2
3 • SHOW TABLES;
```

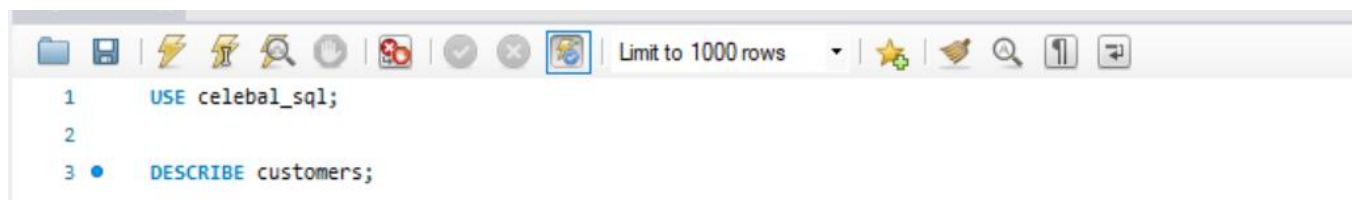
Output Screenshot



Tables_in_celebal_sql
customers
order_items
orders
products

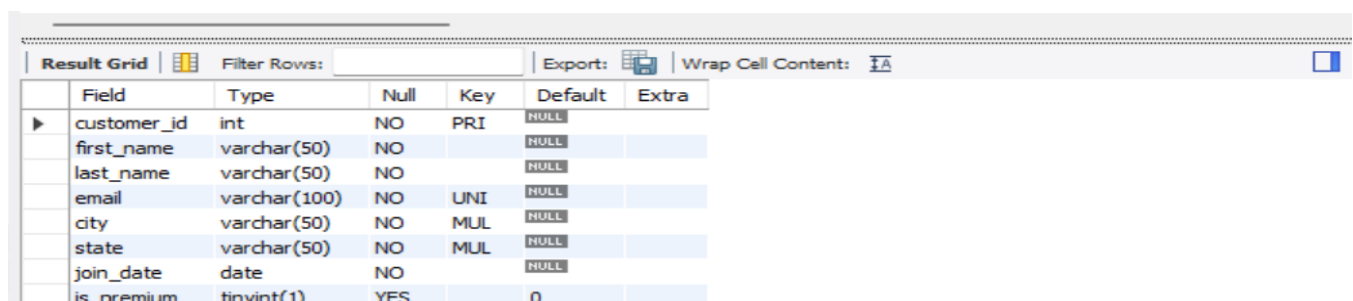
Describe customers

Query Screenshot



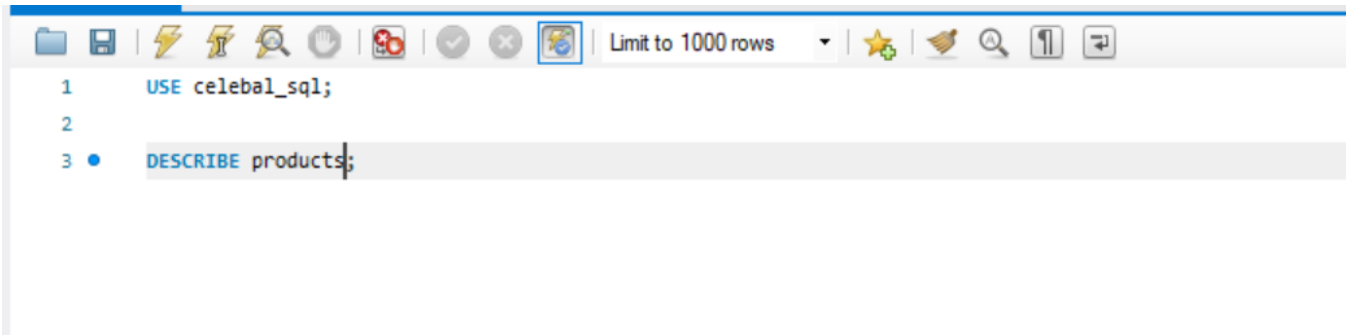
```
1 USE celebal_sql;
2
3 • DESCRIBE customers;
```

Output Screenshot



Field	Type	Null	Key	Default	Extra
customer_id	int	NO	PRI	NULL	
first_name	varchar(50)	NO		NULL	
last_name	varchar(50)	NO		NULL	
email	varchar(100)	NO	UNI	NULL	
city	varchar(50)	NO	MUL	NULL	
state	varchar(50)	NO	MUL	NULL	
join_date	date	NO		NULL	
is_premium	tinyint(1)	YES		0	

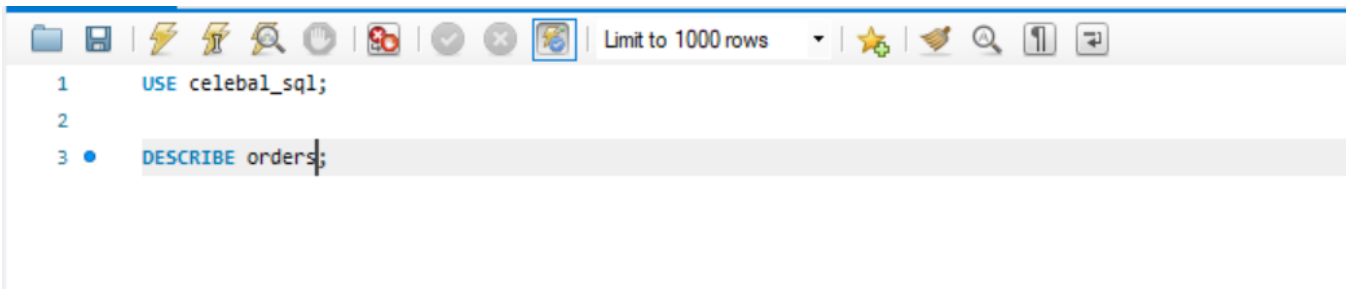
Describe products
Query Screenshot



Output Screenshot

Result Grid						
Filter Rows:		Export:		Wrap Cell Content:		
	Field	Type	Null	Key	Default	Extra
▶	product_id	int	NO	PRI	NULL	
	product_name	varchar(100)	NO		NULL	
	category	varchar(50)	NO	MUL	NULL	
	brand	varchar(50)	NO		NULL	
	unit_price	decimal(10,2)	NO		NULL	
	stock_qty	int	NO		0	

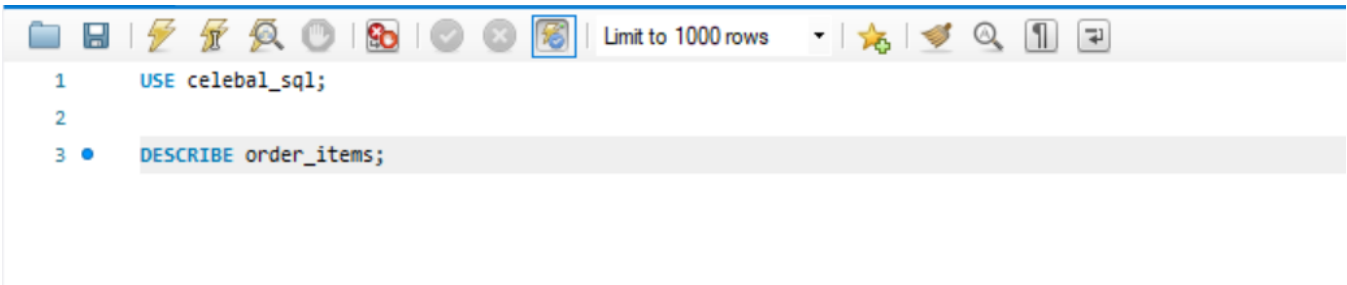
Describe orders
Query Screenshot



Output Screenshot

Result Grid						
Filter Rows:		Export:		Wrap Cell Content:		
	Field	Type	Null	Key	Default	Extra
▶	order_id	int	NO	PRI	NULL	
	customer_id	int	NO	MUL	NULL	
	order_date	date	NO	MUL	NULL	
	status	varchar(20)	NO	MUL	Pending	
	total_amount	decimal(12,2)	NO		NULL	

Describe order_items
Query Screenshot

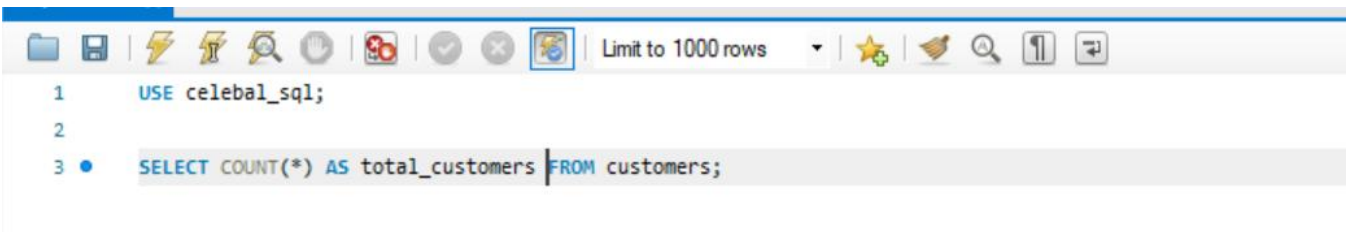


Output Screenshot

Result Grid						
Filter Rows:		Export:		Wrap Cell Content: IA		
	Field	Type	Null	Key	Default	Extra
▶	item_id	int	NO	PRI	NULL	
	order_id	int	NO	MUL	NULL	
	product_id	int	NO	MUL	NULL	
	quantity	int	NO		NULL	
	unit_price	decimal(10,2)	NO		NULL	
	discount_pct	decimal(5,2)	YES		0.00	

Data Verification

Total Customers
Query Screenshot

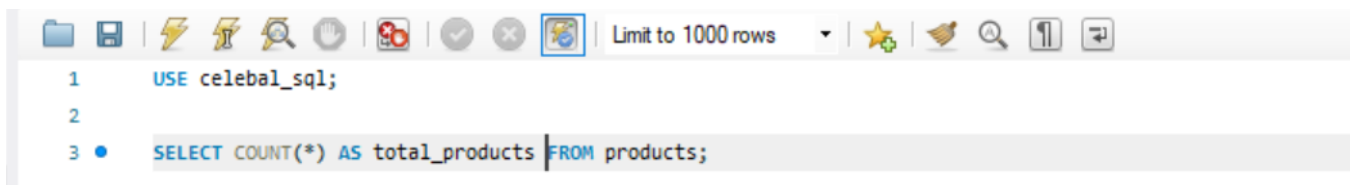


Output Screenshot

Result Grid	
Filter Rows:	
Export:	
Wrap Cell Content: IA	
	total_customers
▶	8

Total Products

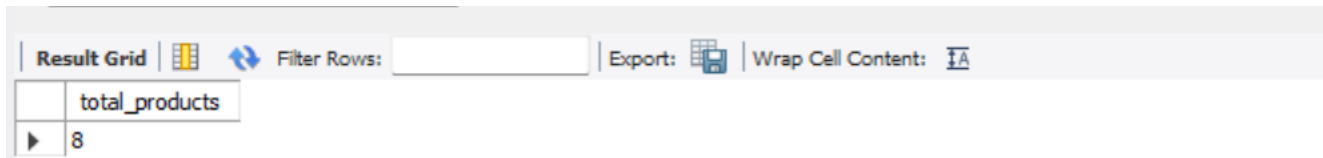
Query Screenshot



The screenshot shows a SQL query editor with a toolbar at the top. The toolbar includes icons for file operations, execution, and a dropdown menu set to "Limit to 1000 rows". The query text is as follows:

```
1 USE celebal_sql;  
2  
3 • SELECT COUNT(*) AS total_products FROM products;
```

Output Screenshot

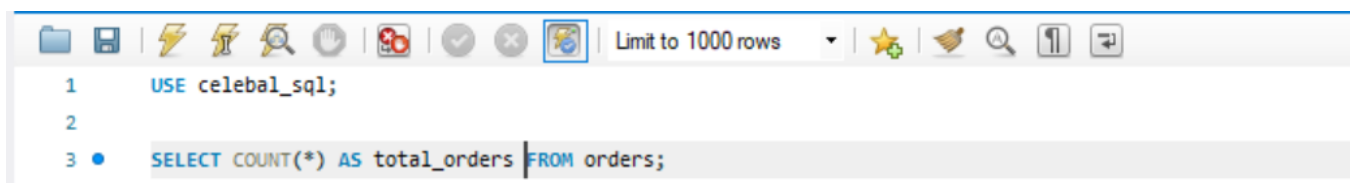


The screenshot shows the output grid of the query. The toolbar at the top includes a "Result Grid" button, a "Filter Rows" input field, an "Export" button, and a "Wrap Cell Content" button. The output is displayed in a table with one column and one row.

	total_products
▶	8

Total Orders

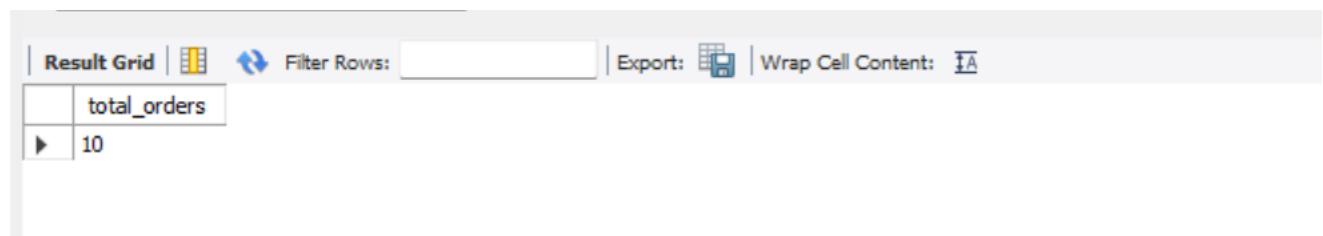
Query Screenshot



The screenshot shows a SQL query editor with a toolbar at the top. The toolbar includes icons for file operations, execution, and a dropdown menu set to "Limit to 1000 rows". The query text is as follows:

```
1 USE celebal_sql;  
2  
3 • SELECT COUNT(*) AS total_orders FROM orders;
```

Output Screenshot

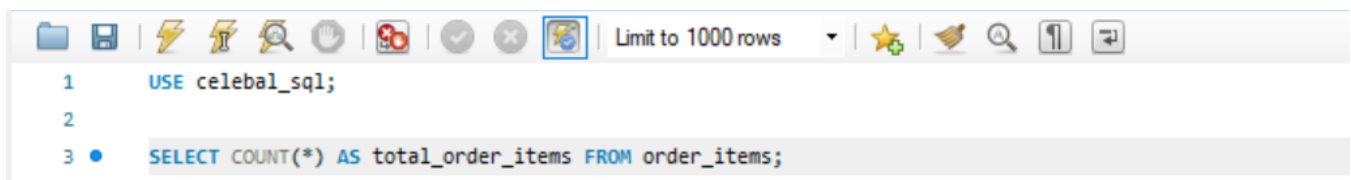


The screenshot shows the output grid of the query. The toolbar at the top includes a "Result Grid" button, a "Filter Rows" input field, an "Export" button, and a "Wrap Cell Content" button. The output is displayed in a table with one column and one row.

	total_orders
▶	10

Total Order Items

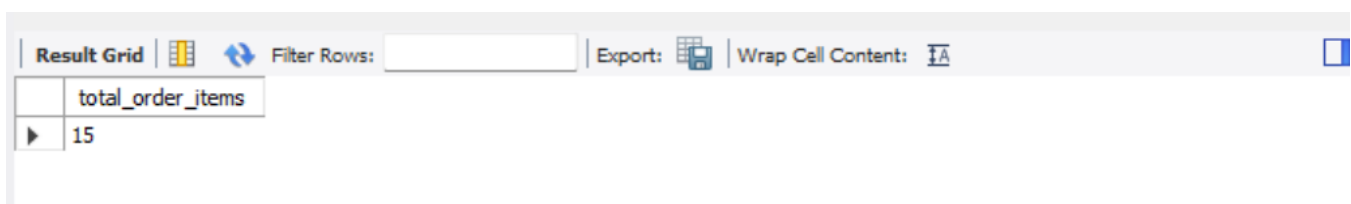
Query Screenshot



The screenshot shows a SQL query editor with a toolbar at the top. The toolbar includes icons for file operations, execution, and a dropdown menu set to "Limit to 1000 rows". The query text is as follows:

```
1 USE celebal_sql;  
2  
3 • SELECT COUNT(*) AS total_order_items FROM order_items;
```

Output Screenshot



The screenshot shows the output grid of the query. The toolbar at the top includes a "Result Grid" button, a "Filter Rows" input field, an "Export" button, and a "Wrap Cell Content" button. The output is displayed in a table with one column and one row.

	total_order_items
▶	15

Section A — SQL Basics (SELECT, Constraints, Primary Keys)

Q1. Write a query to display all columns and rows from the customer's table.

Query –

```
1  USE celebal_sql;
2
3  •  SELECT * FROM customers;
```

Output –

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q2. Retrieve only the first_name, last_name, and city of all customers.

Query –

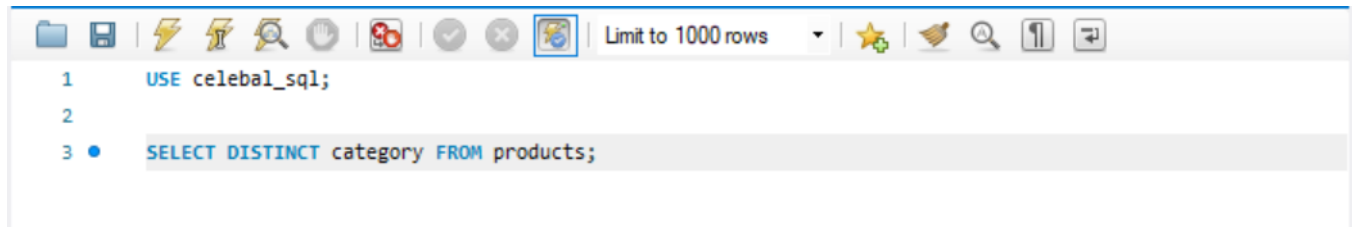
```
1  USE celebal_sql;
2
3  •  SELECT first_name, last_name, city FROM customers;
```

Output –

first_name	last_name	city
Aarav	Sharma	Mumbai
Priya	Patel	Ahmedabad
Rohan	Gupta	Delhi
Sneha	Reddy	Hyderabad
Vikram	Singh	Jaipur
Ananya	Iyer	Chennai
Karan	Mehta	Pune
Divya	Nair	Kochi

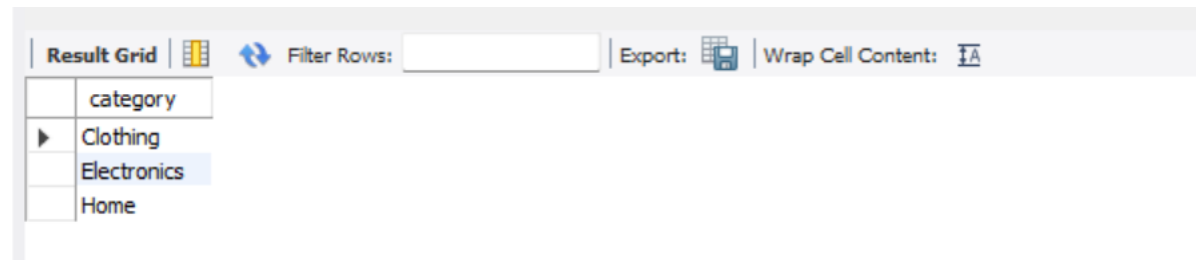
Q3. List all unique categories available in the products table.

Query –



```
1 USE celebal_sql;
2
3 SELECT DISTINCT category FROM products;
```

Output –



	category
▶	Clothing
	Electronics
	Home

Q4. Identify the Primary Key of each table in the schema. Explain why a Primary Key must be unique and NOT NULL.

Answer –

Primary Keys:

1. customers: customer_id
2. products: product_id
3. orders: order_id
4. order_items: item_id

A Primary Key uniquely identifies each record in a table. It cannot contain duplicate values because every record must have a unique identity. It also cannot contain NULL values because every row must always be identifiable.

Q5. What constraints are applied to the email column in the customers table? What would happen if you tried to insert a duplicate email?

Answer –

Constraints applied on email:

- UNIQUE
- NOT NULL

If we try to insert an existing email address, MySQL throws a duplicate entry error because the UNIQUE constraint does not allow duplicate values.

```
1  USE celebal_sql;
2
3  INSERT INTO customers
4  VALUES
5  (
6  109,
7  'Rahul',
8  'Verma',
9  'aarav.s@email.com',
10 'Lucknow',
11 'Uttar Pradesh',
12 '2024-09-01',
13 FALSE
14 );
```

48 13:46:00 INSERT INTO customers VALUES (109, 'Rahul', 'Verma', 'aarav.s@email.com', 'Lucknow', 'Uttar Pradesh', '2024-09-01', FALSE) Error Code: 1062. Duplicate entry 'aarav.s@email.com' for key 'custo...' 0.031 sec

Q6. Try inserting a product with unit_price = -50. What happens and which constraint prevents it? Write both the INSERT statement and explain the error.

Query –

```
1  USE celebal_sql;
2
3  INSERT INTO products (
4  product_id,
5  product_name,
6  category,
7  brand,
8  unit_price,
9  stock_qty
10 )
11 VALUES
12 (
13 209,
14 'USB Cable',
15 'Electronics',
16 'HP',
17 -50,
18 100
19 );
```

50 13:48:51 INSERT INTO products (product_id, product_name, category, brand, unit_price, stock_qty) VALUES (209, 'USB Cable', 'Electronics', 'HP', -50, 100) Error Code: 3819. Check constraint 'products_chk_1' is violated. 0.000 sec

Output –

The INSERT statement fails because the value of unit_price is -50, which violates the CHECK constraint (unit_price > 0). The CHECK constraint ensures that only positive product prices can be stored in the products table.

Section B — Filtering & Optimization (WHERE, Indexes)

Q7. Retrieve all orders with status = 'Delivered'.

Query –

```
1  USE celebal_sql;
2
3  ●  SELECT * FROM orders WHERE status = 'Delivered';
```

Output –

Result Grid	Filter Rows:	Edit:	Export/Import:	Wrap Cell Content:
order_id	customer_id	order_date	status	total_amount
1001	101	2024-08-01	Delivered	4498.00
1002	102	2024-08-03	Delivered	799.00
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1008	103	2024-08-20	Delivered	899.00
1010	108	2024-08-28	Delivered	1598.00
NULL	NULL	NULL	NULL	NULL

Q8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000.

Query –

```
1  USE celebal_sql;
2
3  ●  SELECT * FROM products WHERE category = 'Electronics' AND unit_price > 2000;
```

Output –

Result Grid	Filter Rows:	Edit:	Export/Import:	Wrap Cell Content:
order_id	customer_id	order_date	status	total_amount
1001	101	2024-08-01	Delivered	4498.00
1002	102	2024-08-03	Delivered	799.00
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1008	103	2024-08-20	Delivered	899.00
1010	108	2024-08-28	Delivered	1598.00
NULL	NULL	NULL	NULL	NULL

Q9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.

Query –

```
1  USE celebal_sql;
2
3  •  SELECT * FROM customers WHERE state = 'Maharashtra' AND join_date BETWEEN '2024-01-01' AND '2024-12-31';
```

Output –

Result Grid					
Filter Rows:					
Edit:   					
Export/Import:  					
Wrap Cell Content:  					
	order_id	customer_id	order_date	status	total_amount
▶	1001	101	2024-08-01	Delivered	4498.00
	1002	102	2024-08-03	Delivered	799.00
	1004	101	2024-08-10	Delivered	3499.00
	1006	105	2024-08-15	Delivered	5898.00
	1008	103	2024-08-20	Delivered	899.00
	1010	108	2024-08-28	Delivered	1598.00
*	NULL	NULL	NULL	NULL	NULL

Q10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.

Query –

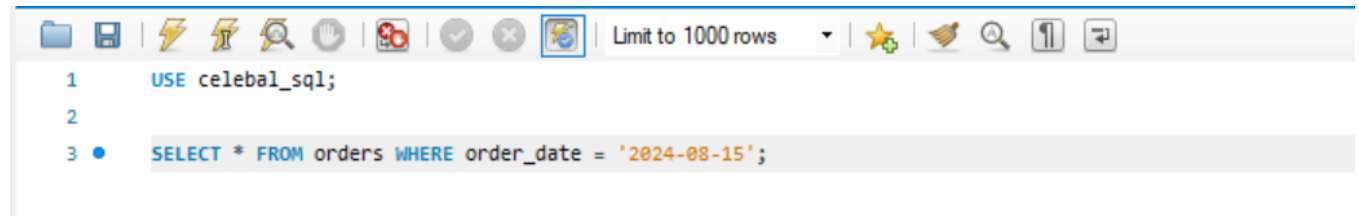
```
1  USE celebal_sql;
2
3  •  SELECT * FROM orders WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25' AND status <> 'Cancelled';
```

Output –

Result Grid					
Filter Rows:					
Edit:   					
Export/Import:  					
Wrap Cell Content:  					
	order_id	customer_id	order_date	status	total_amount
▶	1004	101	2024-08-10	Delivered	3499.00
	1006	105	2024-08-15	Delivered	5898.00
	1007	106	2024-08-18	Pending	1299.00
	1008	103	2024-08-20	Delivered	899.00
	1009	107	2024-08-25	Shipped	6098.00
*	NULL	NULL	NULL	NULL	NULL

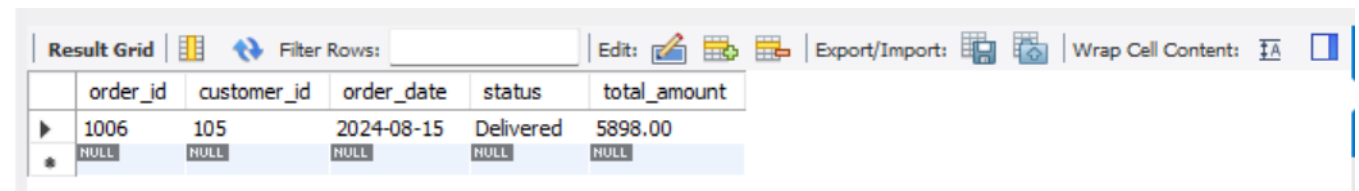
Q11. Explain what the index `idx_orders_date` does. How would it improve the performance of a query that filters orders by `order_date`? Write a sample query that would benefit from this index.

Query –



```
1  USE celebal_sql;
2
3  •  SELECT * FROM orders WHERE order_date = '2024-08-15';
```

Output –



	order_id	customer_id	order_date	status	total_amount
▶	1006	105	2024-08-15	Delivered	5898.00
*	NULL	NULL	NULL	NULL	NULL

The index `idx_orders_date` is created on the `order_date` column.

It helps MySQL locate records based on `order_date` more efficiently without scanning the entire table. This improves query performance, especially when the table contains a large number of records.

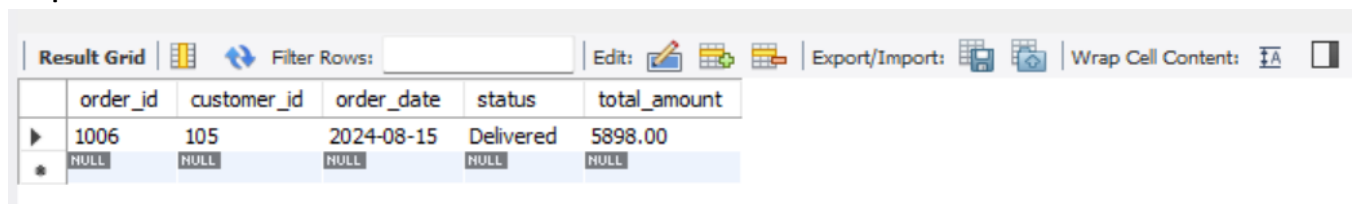
Q12. If you run: `SELECT * FROM customers WHERE YEAR(join_date) = 2024;` — would the index on `join_date` be used? Explain why or why not, and rewrite the query to be index-friendly (SARGable).

Query -



```
1  USE celebal_sql;
2
3  •  SELECT * FROM customers WHERE join_date BETWEEN '2024-01-01' AND '2024-12-31';
```

Output -



	order_id	customer_id	order_date	status	total_amount
▶	1006	105	2024-08-15	Delivered	5898.00
*	NULL	NULL	NULL	NULL	NULL

The query `WHERE YEAR(join_date) = 2024` may not efficiently use an index because the `YEAR()` function is applied to the indexed column. A better approach is to filter using a date range.

Using `BETWEEN` allows MySQL to utilize the index more effectively, making the query faster on large datasets.

Section C — Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)

Q13. Count the total number of orders in the orders table.

Query –

```
1  USE celebal_sql;
2
3  •  SELECT COUNT(*) AS total_orders FROM orders;
```

Output –

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	total_orders			
▶	10			

Q14. Find the total revenue (SUM of total_amount) from all 'Delivered' orders.

Query –

```
1  USE celebal_sql;
2
3  •  SELECT SUM(total_amount) AS total_revenue FROM orders WHERE status = 'Delivered';
```

Output –

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	total_revenue			
▶	17191.00			

Q15. Calculate the average unit_price of products in each category.

Query –

```
1  USE celebal_sql;
2
3  •  SELECT category, AVG(unit_price) AS average_price FROM products GROUP BY category;
```

Output –

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	category	average_price		
▶	Clothing	2699.000000		
	Electronics	2224.000000		
	Home	949.000000		

Q16. For each order status, find the count of orders and the total revenue. Sort the result by total revenue in descending order.

Query –

```
1  USE celebal_sql;
2
3  •  SELECT status, COUNT(order_id) AS total_orders, SUM(total_amount) AS total_revenue FROM orders GROUP BY status ORDER BY total_revenue DESC;
```

Output –

status	total_orders	total_revenue
Delivered	6	17191.00
Shipped	2	13596.00
Cancelled	1	2999.00
Pending	1	1299.00

Q17. Find the most expensive (MAX) and cheapest (MIN) product in each category.

Query –

```
1  USE celebal_sql;
2
3  •  SELECT category, MAX(unit_price) AS highest_price, MIN(unit_price) AS lowest_price FROM products GROUP BY category;
```

Output –

category	highest_price	lowest_price
Clothing	4599.00	799.00
Electronics	3499.00	899.00
Home	1299.00	599.00

Q18. List all product categories where the average unit_price is greater than ₹2000. (Hint: Use HAVING clause)

Query -

```
1  USE celebal_sql;
2
3  •  SELECT category, AVG(unit_price) AS average_price FROM products GROUP BY category HAVING AVG(unit_price) > 2000;
```

Output -

category	average_price
Clothing	2699.000000
Electronics	2224.000000

Section D — Joins & Relationships

Q19. Write an INNER JOIN query to display each order along with the customer's first_name and last_name. Show: order_id, order_date, first_name, last_name, total_amount.

Query –

```
1  USE celebal_sql;
2
3  •  SELECT o.order_id,
4         o.order_date,
5         c.first_name,
6         c.last_name,
7         o.total_amount
8  FROM orders o
9  INNER JOIN customers c
10 ON o.customer_id = c.customer_id;
```

Output –

order_id	order_date	first_name	last_name	total_amount
1001	2024-08-01	Aarav	Sharma	4498.00
1004	2024-08-10	Aarav	Sharma	3499.00
1002	2024-08-03	Priya	Patel	799.00
1003	2024-08-05	Rohan	Gupta	7498.00
1008	2024-08-20	Rohan	Gupta	899.00
1005	2024-08-12	Sneha	Reddy	2999.00
1006	2024-08-15	Vikram	Singh	5898.00
1007	2024-08-18	Ananya	Iyer	1299.00
1009	2024-08-25	Karan	Mehta	6098.00
1010	2024-08-28	Divya	Nair	1598.00

Q20. Using a LEFT JOIN, list ALL customers and their orders (if any). Customers with no orders should still appear with NULL values for order columns.

Query –

```
1  USE celebal_sql;
2
3  •  SELECT c.customer_id,
4         c.first_name,
5         c.last_name,
6         o.order_id,
7         o.order_date,
8         o.status
9  FROM customers c
10 LEFT JOIN orders o
11 ON c.customer_id = o.customer_id;
```

Output –

	customer_id	first_name	last_name	order_id	order_date	status
▶	101	Aarav	Sharma	1001	2024-08-01	Delivered
	101	Aarav	Sharma	1004	2024-08-10	Delivered
	102	Priya	Patel	1002	2024-08-03	Delivered
	103	Rohan	Gupta	1003	2024-08-05	Shipped
	103	Rohan	Gupta	1008	2024-08-20	Delivered
	104	Sneha	Reddy	1005	2024-08-12	Cancelled
	105	Vikram	Singh	1006	2024-08-15	Delivered
	106	Ananya	Iyer	1007	2024-08-18	Pending
	107	Karan	Mehta	1009	2024-08-25	Shipped
	108	Divya	Nair	1010	2024-08-28	Delivered

Q21. Write a query using JOINS across three tables (orders → order_items → products) to show: order_id, product_name, quantity, unit_price, and discount_pct for each order item.

Query –

```
1  USE celestial_sql;
2
3  •  SELECT o.order_id,
4         p.product_name,
5         oi.quantity,
6         oi.unit_price,
7         oi.discount_pct
8  FROM orders o
9  INNER JOIN order_items oi
10 ON o.order_id = oi.order_id
11 INNER JOIN products p
12 ON oi.product_id = p.product_id;
```

Output –

	order_id	product_name	quantity	unit_price	discount_pct
▶	1001	Wireless Earbuds	2	1499.00	0.00
	1006	Wireless Earbuds	1	1499.00	10.00
	1002	Cotton T-Shirt	1	799.00	0.00
	1003	Smart Watch	1	2999.00	0.00
	1005	Smart Watch	1	2999.00	0.00
	1003	Running Shoes	1	4599.00	5.00
	1006	Running Shoes	1	4599.00	5.00
	1004	Bluetooth Speaker	1	3499.00	0.00
	1009	Bluetooth Speaker	1	3499.00	0.00
	1007	Bedsheet Set	1	1299.00	0.00
	1010	Bedsheet Set	1	1299.00	0.00
	1001	Laptop Stand	1	899.00	10.00
	1008	Laptop Stand	1	899.00	0.00
	1009	Cushion Covers (...)	2	599.00	15.00
	1010	Cushion Covers (...)	1	599.00	0.00

Q22. Explain the difference between LEFT JOIN and RIGHT JOIN with an example from this schema. When would you use a FULL OUTER JOIN?

Answer –

LEFT JOIN – Returns all records from the left table and matching records from the right table. If there is no match, NULL values are returned for the right table.

Example: customers LEFT JOIN orders

RIGHT JOIN – Returns all records from the right table and matching records from the left table. If there is no match, NULL values are returned for the left table.

Example: customers RIGHT JOIN orders

FULL OUTER JOIN – Returns all matching and non-matching records from both tables. MySQL does not directly support FULL OUTER JOIN, but it can be achieved using UNION of LEFT JOIN and RIGHT JOIN.

Q23. Identify all Foreign Key relationships in the schema. Explain what would happen if you tried to insert an order with customer_id = 999 (which doesn't exist in customers).

Query -

```
1  USE celebal_sql;
2
3  INSERT INTO orders
4  VALUES
5  (
6    1011,
7    999,
8    '2024-09-01',
9    'Pending',
10   2500.00
11  );
```

Output -

```
78 14:32:14 INSERT INTO orders VALUES (1011, 999, '2024-09-01', 'Pending', 2500.00) Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails ('celebal_sql'. 'order... 0.047 sec
```

Foreign Key Relationships

1. orders.customer_id – customers.customer_id
2. order_items.order_id – orders.order_id
3. order_items.product_id – products.product_id

If an order is inserted with customer_id = 999, MySQL will generate a Foreign Key Constraint Error because customer_id 999 does not exist in the customers table.

Section E — Advanced Concepts (CASE, ACID, Transactions)

These questions test your understanding of conditional logic, database reliability principles, and transaction management.

Q24. Write a query using CASE to classify products into price tiers:

- 'Budget' → unit_price < 1000
- 'Mid-Range' → unit_price BETWEEN 1000 AND 3000
- 'Premium' → unit_price > 3000

Display: product_name, unit_price, price_tier.

Query –

```
1  USE celebal_sql;
2
3  • SELECT product_name,
4          unit_price,
5          CASE
6              WHEN unit_price < 1000 THEN 'Budget'
7              WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'
8              ELSE 'Premium'
9          END AS price_tier
10 FROM products;
```

Output –

product_name	unit_price	price_tier
Wireless Earbuds	1499.00	Mid-Range
Cotton T-Shirt	799.00	Budget
Smart Watch	2999.00	Mid-Range
Running Shoes	4599.00	Premium
Bluetooth Speaker	3499.00	Premium
Bedsheet Set	1299.00	Mid-Range
Laptop Stand	899.00	Budget
Cushion Covers (Set)	599.00	Budget

Q25. Using a CASE statement inside an aggregate function, count how many orders are 'Delivered' vs 'Not Delivered' (all other statuses). Display the result in a single row.

Query –

```
1  USE celebal_sql;
2
3  • SELECT
4      SUM(CASE WHEN status = 'Delivered' THEN 1 ELSE 0 END) AS delivered_orders,
5      SUM(CASE WHEN status <> 'Delivered' THEN 1 ELSE 0 END) AS not_delivered_orders
6  FROM orders;
```

Output –

delivered_orders	not_delivered_orders
6	4

Q26. Explain each letter of ACID:

- A – Atomicity
- C – Consistency
- I – Isolation
- D – Durability

Give a real-world example (e.g., bank transfer) showing why each property is important.

Answer –

A – **Atomicity** – Atomicity ensures that all operations in a transaction are completed successfully. If any operation fails, the entire transaction is rolled back.

C – **Consistency** – Consistency ensures that the database remains valid before and after every transaction by following all rules and constraints.

I – **Isolation** – Isolation ensures that multiple transactions do not interfere with each other while executing simultaneously.

D – **Durability** – Durability guarantees that once a transaction is committed, the changes remain permanently saved even if the system crashes.

Example: In a bank transfer, money is deducted from one account and added to another. If the system crashes after deducting the amount but before adding it to the receiver's account, the transaction is rolled back. This maintains Atomicity, Consistency, Isolation, and Durability.

Q27. Write a SQL transaction that does the following atomically:

1. Insert a new order (order_id=1011, customer_id=102, today's date, 'Pending', 1598.00)
2. Insert two order items for that order
3. Update the stock_qty of the purchased products
4. If any step fails, ROLLBACK the entire transaction. Otherwise, COMMIT.

Write the complete BEGIN...COMMIT/ROLLBACK block.

Query –

```
1  USE celebal_sql;
2
3  START TRANSACTION;
4
5  INSERT INTO orders
6  VALUES(1011,102,CURDATE(),'Pending',1598.00);
7  INSERT INTO order_items
8  VALUES(5016,1011,206,1,1299.00,0);
9  INSERT INTO order_items
10 VALUES(5017,1011,208,1,299.00,0);
11
12 UPDATE products
13 SET stock_qty = stock_qty - 1
14 WHERE product_id = 206;
15
16 UPDATE products
17 SET stock_qty = stock_qty - 1
18 WHERE product_id = 208;
19
20 COMMIT;
21
```

Output –

84	14:43:07	START TRANSACTION	0 row(s) affected	0.000 sec
85	14:43:07	INSERT INTO orders VALUES (1011, 102, CURDATE(), 'Pending', 1598.00)	1 row(s) affected	0.032 sec
86	14:43:07	INSERT INTO order_items VALUES (5016, 1011, 206, 1, 1299.00, 0)	1 row(s) affected	0.000 sec
87	14:43:07	INSERT INTO order_items VALUES (5017, 1011, 208, 1, 299.00, 0)	1 row(s) affected	0.000 sec
88	14:43:07	UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 206	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.016 sec
89	14:43:07	UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 208	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
90	14:43:07	COMMIT	0 row(s) affected	0.016 sec